UNIVERSITAT POLITÈCNICA DE VALÈNCIA

DSIC

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

Dpto. de Sistemas Informáticos y Computación

Tipos Cociente en un Lenguaje Dependientemente Tipado

Trabajo Fin de Máster

Máster Universitario en Ingeniería y Tecnología de Sistemas Software

AUTOR: Vegas, Luis

Tutor: Escobar Román, Santiago

CURSO ACADÉMICO: 2025-2026

Índice general

1. Introducción **1**
1.1. Motivación 1
1.2. Objetivos 1
1.3. Estructura del la tesis 1

2. Fundamentos teóricos **2**
2.1. El Lambda cálculo 2
2.1.1. β-reducción 2
2.1.2. Variables libres y ligadas 3
2.1.3. Sustitución 3
2.2. Teoria de tipos 3
2.3. Lambda cálculo simplemente tipado 3
2.4. Tipos dependientes 3
2.5. Teoria de tipos de Martin-Löf 3
2.6. Reescritura de terminos en Maude 3
2.7. Tipos Cociente 3

3. Un lenguaje dependientemente tipado **4**
3.1. Sintaxis 4
3.2. Sustitucion 4
3.3. Normalizacion 4
3.4. Verificacion de tipos 4

4. Extension con tipos cociente **5**
4.1. Formacion 5
4.2. Introduccion 5
4.3. Eliminacion 5
4.4. Computacion 5

5. Conclusiones **6**

A. Código Maude **7**

Capítulo 1

Introducción

1.1. Motivación

1.2. Objetivos

1.3. Estructura del la tesis

Capítulo 2

Fundamentos teóricos

2.1. El Lambda cálculo

El lambda cálculo fue inventado por Alonzo Church con el fin de capturar la nocion abstracta de la aplicabilidad de una funcion. Posteriormente, fue adoptado como un sistema formal de lógica matemática, pues con un conjunto mínimo de reglas es posible modelar y estudiar cómo se computan las funciones matemáticas dada una teoría.

Este sistema es ampliamente utilizado en el diseño de lenguajes de programación. De forma análoga, la forma en la que se programa en el mismo es muy similar a cómo se programa en un lenguaje funcional hoy en dia. Lenguajes como Haskell, Agda, la familia de lenguajes basados en ML, y LISP, están ampliamente inspirados en un pequeño conjunto del cálculo lambda.

El lambda calculo es un lenguaje muy potente y, a su vez, sumamente simple. Un término puede estar compuesto, o bien, formarse, de tres elementos: variables, aplicacion y abstracción lambda. De forma inductiva, su sintaxis se describe como sigue:

$$\begin{aligned} M := \quad & x && (1)\\ | \quad & \lambda x.\, M && (2)\\ | \quad & M\ N && (3) \end{aligned}$$

Donde, siendo Λ el conjunto de términos lambda válidos, (1) es un conjunto infinito de variables, tal que si $x, y, z \in V$ y $V \in \Lambda$, entonces $V \in \Lambda$. (2) es una abstracción lambda, si $x \in V$ y además $M \in \Lambda$, entonces $\lambda x.\, M \in \Lambda$. (3) Es una aplicación de dos términos, si $M, N \in \Lambda$ entonces $MN \in \Lambda$.

A continuación, se detallan propiedades de sumo interés que, posteriormente, formarán parte de los fundamentos en los que el lenguaje que se describe en esta tésis, están basados.

2.1.1. β-reducción

La noción de evaluacion de un término viene dada por beta-reducción, donde la notación $\rightarrow_\beta$ es empleada. La $\rightarrow_\beta$ está basada en la *sustitución*, formalmente expresada como *M[x := N]*, significando que M es un término donde x ha sido reemplazado por N.

Por tanto, una expresión de la forma $(\lambda x.\, M)\, N$ puede ser reescrita como *M[x := N]*, donde toda ocurrencia de x en M (estando x libre en M), debe ser reemplazada por N (German Geuvers, 2014).

Ejemplo 2.1. $(\lambda x.\, x + 1)\, 5$ reduce a $x[x := 5] = 6$

2.1.2. Variables libres y ligadas

2.1.3. Sustitución

Anteriormente, definimos la sustitución en términos de $\rightarrow_\beta$ reducción, sin embargo, la sustitucion posee en si reglas de computo que valen la pena considerar pues, dependiendo del término que se esté evaluando, la sustitución tendrá, o no, efecto del todo.

Se definen, pues, las reglas de computo de la sustitucion de acuerdo a lo discutido por Pierce (2002). Más formalmente:

(1) $x[x := N] \equiv N$,
(2) $y[x := N] \equiv y$ if $x \not\equiv y$,
(3) $(PQ)[x := N] \equiv (P[x := N])(Q[x := N])$,
(4) $(\lambda y.\, P)[x := N] \equiv \lambda z.\, (P^{y \rightarrow z}[x := N])$, if $\lambda z.\, P^{y \rightarrow z}$ is an α-variant of $\lambda y.\, P$ such that $z \notin FV(N)$.

Para (1), sustituyendo x, por N dentro de x, siendo x el termino que buscamos sustituir, obtenemos N. De forma analoga, sustituyendo cada ocurrencia de x en el termino y, por N, si x e y no son iguales, el termino intacto, y, sera retornado. El caso de (3), tiene sentido cuando P es una abstraccion aplicada a un termino Q. Concretamente, se reduce a la sustitucion de x por N, en ambos P y Q. (4) —->TODO

2.2. Teoria de tipos

2.3. Lambda cálculo simplemente tipado

2.4. Tipos dependientes

2.5. Teoria de tipos de Martin-Löf

2.6. Reescritura de terminos en Maude

2.7. Tipos Cociente

Capítulo 3

Un lenguaje dependientemente tipado

3.1. Sintaxis

3.2. Sustitucion

3.3. Normalizacion

3.4. Verificacion de tipos

Capítulo 4

Extension con tipos cociente

4.1. Formacion

4.2. Introduccion

4.3. Eliminacion

4.4. Computacion

Capítulo 5

Conclusiones

Apéndice A

Código Maude

TODO

Bibliografía

Rob Nederpelt German Geuvers. *Type Theory and Formal Proof.* Cambride University Press, 2014.

Ulf Norell. *Towards a Practical Programming Language Based on Dependent Type Theory.* PhD thesis, Chalmers University of Technology, 2007.

Benjamin C. Pierce. *Types and Programming Languages.* MIT Press, 2002.